Assignment 4

1. What are environment variables?

Environment variables are variables maintained by the operating system and the shell. They store information that can be used by programs and commands running in the system.

Some common environment variables are:`PATH`,`HOME`,`USER`,`SHELL`,`PWD`

a. Do some environment variables have a scope similar to global and local variables?

Yes. Environment variables can have different scopes.

- System-wide environment variables are available to multiple or all users on the system. They are similar to global variables.
- User-specific environment variables are available only to a particular user. They are similar to local variables in terms of scope.

For example, system-wide settings can be configured in files such as:

```
/etc/environment
/etc/profile
```

User-specific settings are commonly configured in files such as:

```
~/.profile
~/.bashrc
```

b. How can one find out the environment variables available to all users?

System-wide environment variables can be checked in:`cat /etc/environment`

You can also examine:`cat /etc/profile`

The `/etc/environment` file is commonly used to define environment variables that apply system-wide.

c. How can one find out the environment variables set specifically for a user? How do you set them?

The environment variables available to the current user can be displayed using:`env`

A specific variable can be displayed using: `echo $HOME`

To create a user-specific environment variable, the `export` command can be used:

```
export COLLEGE="SRM"
```

The value can be checked using : `echo $COLLEGE`

Output : `SRM`

This variable is available in the current shell session. To make it permanent for the user, the following line can be added to the user's `~/.bashrc` file:

```
export COLLEGE="SRM"
```

After saving the file, use:

`source ~/.bashrc` ,to apply the changes.

2. Write a shell program that prints the numbers from 1 to 100 with each line having the number in digits, followed by a space and then the number in words.

```
#!/bin/bash

ones=("" "one" "two" "three" "four" "five" "six" "seven" "eight"
"nine")
teens=("ten" "eleven" "twelve" "thirteen" "fourteen" "fifteen"
"sixteen" "seventeen" "eighteen" "nineteen")
tens=("" "" "twenty" "thirty" "forty" "fifty" "sixty" "seventy"
"eighty" "ninety")

for ((i=1; i<=100; i++))
do
    if [ $i -lt 10 ]; then
        word=${ones[$i]}
    elif [ $i -lt 20 ]; then
        word=${teens[$((i-10))]}
    elif [ $i -lt 100 ]; then
        if [ $((i % 10)) -eq 0 ]; then
            word=${tens[$((i/10))]}
        else
            word="${tens[$((i/10))]} ${ones[$((i%10))]}"
        fi
    else
        word="hundred"
    fi

    echo "$i $word"
Done
```

3. How many unique processes are currently running on your Linux system?

The `ps` command can be used to list the currently running processes.

To display the command names: `ps -eo cmd`

To find the number of unique commands: `ps -eo cmd | sort | uniq | wc -l`

Explanation

- `ps -eo cmd` → displays the command name of each process.
- `sort` → sorts the output.
- `uniq` → removes duplicate entries.
- `wc -l` → counts the number of lines.

Another solution using the `cut` command is:

```
ps -eo pid,cmd | cut -d' ' -f2 | sort | uniq | wc -l
```

The exact number will depend on the processes currently running on the Linux system, so it may be different each time the command is executed.

4.Write a shell script that takes 2 integers as command line arguments and prints their sum. Does your script work if one or both of the integers are negative numbers or zero? If not, modify your program or the way you run the script to make it work.

Create a file called `sum.sh`:

```
#!/bin/bash
sum=$(($1 + $2))
echo "Sum = $sum"
```

Give it execute permission: `chmod +x sum.sh`

Run the script with two integers: `./sum.sh 10 20`

Output: `Sum = 30`

Negative numbers and zero

The script also works with zero and negative numbers.

For example:

```
./sum.sh -10 20
```

Output: `Sum = 10`

For two negative numbers: `./sum.sh -10 -20`

Output: `Sum = -30`

For zero: `./sum.sh 0 20`

Output: `Sum = 20`

When using negative numbers, putting a space between the command and each argument is important. The negative sign is treated as part of the argument.

5. Explore the grep command. Can you find an alternative for wc -l, using grep?

Yes. The `grep` command can be used as an alternative to `wc -l` for counting lines.

```
grep -c . filename
```

This counts the lines in `filename` that contain at least one character.

Another example:

```
cat filename | grep -c .
```

Here:

```
grep -c .
```

counts the matching lines.

For a file containing no empty lines, this can be used as an alternative to:

```
wc -l filename
```

However, `wc -l` is the more direct command for counting all lines, including empty lines.

6. What commands (or combination of commands) can be used to pipe the output of the program in Question 2 so that only the lines containing 2-digit numbers (i.e., 10 to 99) are printed?

The output of the program in Question 2 can be piped to the grep command to select only the lines containing two-digit numbers.

If the program is saved as `numbers.sh`, use:

```
./numbers.sh | grep -E '^[0-9]{2} '
```

Explanation

- `./numbers.sh` → runs the program from Question 2.
- `|` → sends the output of the program to the next command.
- `grep -E` → searches using an extended regular expression.
- `^[0-9]{2}` → matches exactly two digits at the beginning of a line.
- The space after `{2}` ensures that the two digits form the complete number.